

Jet clustering algorithms in the FCC jet benchmarks

Distance measures, termination rules, and what the method directories contain

FCCJetBenchmarks

September 12, 2026

Colour coding follows the repository figures:

■ Durham / PFJets ■ e^+e^- C/A, $p = 0$ ■ e^+e^- k_T , $p = +1$ ■ e^+e^- anti- k_T , $p = -1$ ■ CaloJets

Common framework

All jets are built with FastJet through the FCCAnalyses JetClustering functors

Every algorithm below is a sequential recombination algorithm: take the smallest of all d_{ij} and d_{iB} , merge the pair ij or declare i a jet, repeat. The algorithms differ only in the distance measure and in the termination rule.

Setting	Value
Collision	e^+e^- at $\sqrt{s} = 240$ GeV, ZH production
Recombination scheme	E -scheme, four-momenta added
Gen-jet input	stable generator particles, neutrinos removed
Reco-jet input	particle-flow objects
Acceptance	$ \eta < 2.56$
Radius grid	$R = 0.4, 0.6, 0.8, 1.0, 1.2, 1.4$
Jets per process	$N = 2, 4$ or 6

The radius grid and N are defined by `RADIUS_SCAN` and `NUMBER_OF_JETS` in `src/process_config.py`.

Distance measures side by side

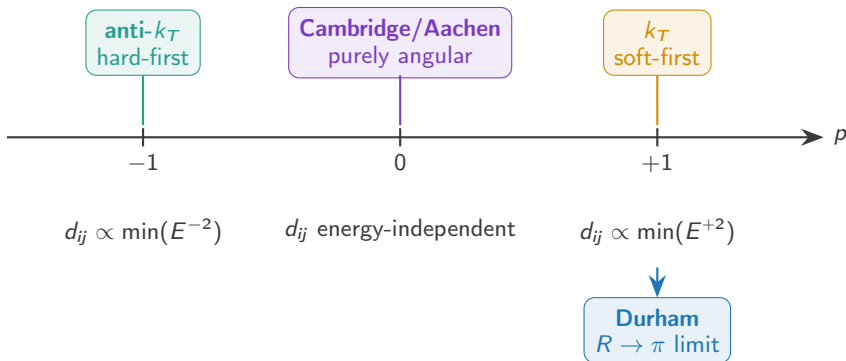
Algorithm	d_{ij}	d_{iB}	Termination
generalized $e^+e^- \ k_T$	$\min\left(E_i^{2p}, E_j^{2p}\right) \frac{1 - \cos \theta_{ij}}{1 - \cos R}$	E_i^{2p}	inclusive
Durham $(e^+e^- \ k_T)$	$\frac{2 \min(E_i^2, E_j^2) (1 - \cos \theta_{ij})}{E_{\text{vis}}^2}$	—	exclusive, exactly N
hadron-collider anti- k_T	$\min\left(p_{T,i}^{-2}, p_{T,j}^{-2}\right) \frac{\Delta R_{ij}^2}{R^2}$	$p_{T,i}^{-2}$	inclusive

Specialisations of the first row: $p = -1$, anti- k_T $p = 0$, C/A $p = +1$, k_T

The e^+e^- measures use angular, not rapidity, distances: at a lepton collider there is no beam remnant to be boosted along. The hadron-collider measure is reachable via `--jet-algorithm AK` and is kept for cross-checks only; it enters no production figure.

The exponent p selects the algorithm

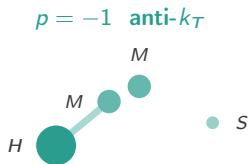
FastJet ee_genkt_algorithm; sixth argument of clustering_ee_genkt



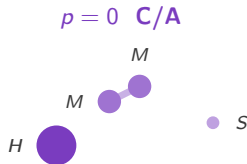
Method directories: **PF_EEAntiKtR*** ($p = -1$), **PF_EECambridgeR*** ($p = 0$), **PF_EEKtR*** ($p = +1$), each spanning the six radii.

Which pair merges first

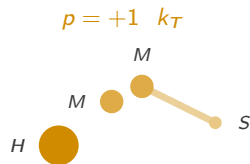
Identical configuration in all three panels; marker size indicates energy



pairs containing a hard particle
are favoured: H absorbs its neighbour



energy drops out:
the smallest angle wins



pairs containing a soft particle
are favoured: S is absorbed first

The weight $\min(E_i^{2p}, E_j^{2p})$ reorders the pairs at fixed geometry, so p fixes the merge sequence and thereby the jet substructure.

Two consequences of the e^+e^- measure

R acts only on the beam comparison

- $1/(1 - \cos R)$ is a global constant and never reorders the d_{ij} among themselves.
- The whole R dependence enters through d_{ij} versus d_{iB} , i.e. through which particles are split off rather than merged.
- Small R therefore yields more, narrower jets.

$p = +1$ degenerates into Durham

- As $R \rightarrow \pi$ the beam term can no longer win. `JetDefinition.hh` states that $R > 2$ with $p = 1$ gives `ee_kt`.
- Measured on simulated 6-jet events: exclusive-to- N clustering with $p = +1$ is bit-identical to Durham for $R \gtrsim 0.6$.
- Consequence: the k_T family is run *inclusively*, where R genuinely changes the jets. An exclusive scan would reproduce [PF_Durham](#).

Two entries that are not clustering algorithms

CaloJets — the no-particle-flow reference

Jets are read from the CaloJetDurham branch produced by Delphes, which applies Durham to calorimeter towers rather than to particle-flow objects. No clustering is performed in this framework. Method directory [CaloJets_Durham](#).

Energy recovery — the “-ER” variants

A post-processing step on the inclusive C/A output (ZHfunctions::energy_recovery, src/histmaker_functions/functions.h). The N leading jets are kept and every surplus jet is recombined into them, so the event ends with exactly N jets. Applied to the C/A scan only. There is deliberately no k_T equivalent: energy recovery on a collection that already has exactly N jets is the identity.

Method directories

Method directory	FastJet	p	Mode
PF_Durham	ee_kt	+1	exclusive, N jets
CaloJets_Durham	ee_kt (Delphes)	+1	exclusive; calorimeter towers
PF_Durham_IdealMatching	ee_kt	+1	exclusive; PFlow-replaced constituents
PF_EECambridgeR{04..14}	ee_genkt	0	inclusive
PF_E_recovery_EECambridgeR{04..14}	ee_genkt	0	inclusive + energy recovery
PF_EEKtR{04..14}	ee_genkt	+1	inclusive
PF_EEAktR{04..14}	ee_genkt	-1	inclusive

The Durham call is `JetClustering::clustering_ee_kt(2, N, 1, 0)`, with `exclusive = 2` meaning “exactly cut jets” and E -ordered output.

Measured behaviour at $R = 0.8$

$Z(\rightarrow q\bar{q}) H(\rightarrow WW \rightarrow q\bar{q}q\bar{q})$, *six expected jets*

Algorithm	Events yielding 6 jets	Fully-matched-event pass rate
$e^+e^- k_T$ ($p = +1$)	74%	0.61
e^+e^- C/A ($p = 0$)	98%	0.82
Durham (exclusive)	—	0.87

- At equal R , k_T merges considerably more aggressively than C/A, and the lost jets propagate into the matching efficiency.
- Durham imposes N , so it has no jet-multiplicity distribution and no radius to scan.

Naming correction, 2026-09

Required for reading any earlier figure or table

Until 2026-09 the $p = 0$ scan was labelled “generalized e^+e^- anti- k_T ” and its directories were named PF_AntiKtR*. That label was wrong. The code called

```
clustering_ee_genkt(R, 0, 0, 0)
```

with four arguments, whereas the signature is

```
(radius, exclusive, cut, sorted, recombination=0, exponent=0.)
```

The exponent defaulted to 0, so those runs were Cambridge/Aachen and never anti- k_T .

- The histograms are valid C/A data; they were relabelled rather than re-run. Directories are now PF_EECambridgeR*.
- Genuine e^+e^- anti- k_T ($p = -1$) is being run for the first time now, as PF_EEAntiKtR*.
- Any older figure showing “ee-AK”, “anti-kt” or AntiKtR must be read as Cambridge/Aachen.